

Rankless: Data-Specific Compilation for Real-Time Citation Network Exploration

Endre M. Borza*

August 15, 2026

Abstract

Interactive exploration of the global scholarly citation network – hundreds of millions of papers, billions of edges – is a highly challenging task: databases are too slow for deep hierarchical breakdowns, while full precomputation is infeasible across millions of queryable entities. We present Rankless, a publicly deployed platform built on *data-specific compilation*, a build-time pipeline that compiles dataset structure into Rust source code and typed binary layouts, eliminating runtime query parsing, planning, and type dispatch, while increasing caching efficiency. In a controlled comparison of 792 hierarchical queries across 5 entity types, Rankless responds $65.3\times$ faster and uses $3.4\times$ less peak memory than a PostgreSQL + Flask baseline. The live system at rankless.org serves interactive visualizations based on this technology from a single commodity server.

Keywords: data visualization; scholarly data; low-latency web systems; metaprogramming; graph analytics

1 Introduction

Exploring how a university’s impact distributes across countries, or which subfields drive an author’s citations, requires navigating hundreds of millions of papers connected by billions of edges. Existing platforms provide search and retrieval [2, 1] or reduce complex profiles to ordinal rankings [?] – none deliver real-time, multi-level hierarchical breakdowns at full dataset scale.

We present Rankless [5], demonstrating that large, mostly-static datasets are more efficiently served by compiling dataset knowledge into the binary than by querying a database at runtime – a web architecture pattern we term *data-specific compilation*. A build-time pipeline generates typed Rust code and binary layouts from the dataset’s structure, the serving layer executes queries over arrays with no query planner, no type dispatch, and no SQL layer – sub-second interactive exploration from a single commodity server, directly addressing ICWE 2026’s focus on performant and sustainable web systems.

2 System Architecture

2.1 Data-Specific Compilation Pipeline

The pipeline ingests OpenAlex [1] JSON dumps and SCImago journal rankings through sequential Rust steps. Each step reads predecessors’ outputs, writes binary data, and generates Rust source via the `dmove_macro` metaprogramming crate – producing entity type definitions, attribute accessors, and link iterators whose types match actual data dimensions. The serving binary encodes every byte layout at compile time; the run phase is a stateless HTTP server.

*Center for Collective Learning, CIAS, Corvinus University, Budapest, Hungary. endre.borza@uni-corvinus.hu

Table 1: Performance comparison over 792 hierarchical breakdown queries.

Metric	PostgreSQL + Flask	Rankless (Rust)	Ratio
Total query time	6 402.3 s	98.0 s	65.3×
Peak memory	4 222 MiB	1 225 MiB	3.4×

2.2 Serving Layer

The Axum HTTP server loads binary data. A Unicode-normalized trie provides sub-millisecond entity search. A thread-pooled tree engine builds multi-level breakdowns on the fly – e.g., impact by field → sub-field → country – surfacing the most-cited paper per branch. Entities with >100k citations are proactively cached. The Svelte frontend renders visualizations as hand-written SVG, with Cytoscape.js [4] for co-authorship layout – the sole dependency.

3 Performance Evaluation

We compare Rankless against a PostgreSQL + Flask baseline serving the same OpenAlex data in a normalized, indexed schema. Both systems run in Docker containers on the same machine over 792 hierarchical queries spanning 5 entity types and 32 breakdown configurations with 1–4 hierarchy levels (Table 1). A full benchmark description is available in the project repository.¹

Rankless completes the full query set in 98.0s – averaging ~124ms per query – versus 6 402.3s for the baseline PostgreSQL. The baseline uses standard best practices (indexes, foreign keys, normalized schema), without query-specific denormalization. Beyond eliminating joins and query planning, the generated code produces functions simple enough to avoid pressure on the L1 instruction cache, and typed layouts use narrow integer widths, reducing data cache pressure. Correctness was validated by comparing breakdown trees from both systems: 20/32 configurations achieve Pearson $r > 0.99$ with mean relative error below 1%.

4 Demonstration Scenario

Attendees interact directly with the live system² using a browser. Each cycle lasts under two minutes and can be repeated for multiple participants:

1. **Personal discovery.** Attendees search for their own institution or themselves as an author – autocomplete responds in under 10ms – and immediately see a research profile.
2. **Hierarchical exploration.** The interactive tree reveals how impact distributes across fields, countries, and institutions: an attendee can discover, for example, which subfield at their university attracts the most international citations.
3. **Live reconfiguration.** Toggling between *production* and *impact*, applying year filters, and switching breakdown axes trigger instant recomputation – demonstrating compiled data access in practice.
4. **Network views.** Research-space networks, geographical citation flows, co-authorship graphs, and author peer heatmaps provide complementary perspectives on any entity.

This dataset covers 1.7B citations, 80M works, 4M authors, and 33k institutions from the OpenAlex snapshot, entities with sufficient citation activity for meaningful breakdowns.

¹<https://github.com/endremborza/rankless/blob/rankless-main/docs/benchmarking>

²<https://rankless.org>

5 Related Work

Semantic Scholar [2] and OpenAlex [1] provide search and API access but optimize for retrieval, not interactive multi-level analytics. Graph databases and OLAP engines support flexible analytical queries but incur runtime query-planning overhead that grows with hierarchy depth. Rankless trades the flexibility of general-purpose query engines for speed and efficiency by compiling dataset structure directly into the binary – a trade-off viable when data is large, mostly static, and queried along known hierarchical dimensions.

6 Conclusion

Data-specific compilation – generating typed code and binaries from dataset characteristics – enables interactive exploration of billion-edge graphs from a single commodity server at $65.3\times$ the throughput of a database baseline. The pattern applies wherever large, mostly-static data is queried along known dimensions: biological interaction networks, knowledge graphs, or web-scale recommendation systems. By replacing a database cluster with a compiled binary, data-specific compilation also offers a concrete, lower-energy web architecture. Future work will focus on structured APIs for autonomous scholarly agents.

Acknowledgments

Supported by the Learn Data ERA Chair from the European Research Executive Agency, funded by the European Union under Horizon Europe Grant Agreement 101086712. The author used generative AI for stylistic editing of the manuscript. The author has no competing interests to declare that are relevant to the content of this article.

References

- [1] Priem, J., Piwowar, H., Orr, R.: OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833 (2022)
- [2] Kinney, R., et al.: The Semantic Scholar open data platform. arXiv:2301.10140 (2023)
- [3] SCImago Journal Rank, <https://www.scimagojr.com> (2026)
- [4] Shannon, P., et al.: Cytoscape: A software environment for integrated models of biomolecular interaction networks. *Genome Research* **13**(11), 2498–2504 (2003)
- [5] Borza, E.M.: Rankless, <https://github.com/endremborza/rankless> (2026)